Taylor's Theorem with Lagrange Remainder in Mathlib
A Dependency-Guided Formal Exposition

Brahimi Mahdi Tahar
University of Mohamed Boudiaf of Msila, Algeria
mahditahar.brahimi@univ-msila.dz

Abstract

We present a dependency-guided exposition of Taylor's theorem with Lagrange remainder as formalised in Mathlib. The paper roots a depth-5 dependency tree at the iterated-derivative statement taylor_mean_remainder_lagrange_iteratedDeriv, shows that it is obtained by combining the scalar Lagrange-remainder theorem taylor_mean_remainder_lagrange with the iterated-derivative machinery of iteratedDerivWithin and the C^n-regularity interface ContDiffOn. At Mathlib commit `0c1d5a7e`, the declaration itself has #print axioms output [propext, Quot.sound]: the proof term does not invoke Classical.choice. We also give a schematic Lean skeleton that mirrors the structure of the Mathlib proof. We distinguish the proof-term axiom closure from the definitional closure of the statement, which does reach Classical.choice through the Real.linearOrder instance — the distinction is precisely what the depth-5 tree makes visible. The paper complements the corpus-scale axiom-profile measurement [1] by exhibiting a concrete, mathematically significant theorem whose proof term sits inside the choice-free corner of Mathlib, while its statement's definitional closure does not. Every numerical claim in the paper is either quoted verbatim from a citable source or derived from those quotes by elementary arithmetic; the sole exception is the direct-use count $D = 206$, labelled explicitly as the authors' own count on the same corpus snapshot (Section 1, provenance table).

2020 Mathematics Subject Classification. Primary 26A06; 26A24. Secondary 41A58; 68V20; 68V15.
Key words and phrases. Taylor's theorem, Lagrange remainder, iterated derivative, Mathlib, Lean 4, dependency tree, formalisation, choice-freeness, proof-term axiom closure.

1. Introduction

Taylor's theorem with Lagrange remainder is the sharpest of the elementary mean-value formulations of the local expansion of a smooth function. In Mathlib [4] it is not a single declaration but a small hierarchy: the scalar mean-remainder statement taylor_mean_remainder_lagrange, its iterated-derivative generalisation taylor_mean_remainder_lagrange_iteratedDeriv that is the root of this paper, the auxiliary evaluation operator taylorWithinEval, and the C^n-regularity interface ContDiffOn. The hierarchy is stable across Mathlib releases and is cited in downstream work on analyticity, mean-value inequalities, and change of variables.

What this paper does. We make three contributions.

(1) A dependency tree. We expand the iterated-derivative Lagrange-remainder theorem to depth 5 using the MathlibGraph edge dataset [8, 7, 9]. The printed excerpt is chosen so that all four kernel-adjacent names reached by the closure — propext, funext, Quot.sound and Classical.choice — appear inside it, and so that the reader can see where each is reached (Section 6.1).

(2) Statement tables and a proof strategy. We collect the fourteen definitions, thirteen lemmas, thirteen theorems, two propositions and two corollaries that make up the hierarchy (Sections 3–4), verify each signature against the Mathlib4 documentation browser [5] and the source file [6], and give the five-step reduction from the iterated statement to the scalar Lagrange form (Section 5).

(3) A Lean skeleton and the proof-term axiom closure. For the Mathlib declaration at commit `0c1d5a7e`, the #print axioms output is {propext, Quot.sound}: the proof term depends only on those two kernel-adjacent names and does not invoke Classical.choice. We also give a Lean 4 schematic skeleton for the iterated theorem that mirrors the structure of the Mathlib proof (Section 6.2).

Choice-freeness: two distinct notions. We must be precise about what "choice-free" means, because the two notions diverge for the theorem under study.

- The proof-term axiom closure of a declaration d is the set of Lean kernel axioms reported by #print axioms d. For taylor_mean_remainder_lagrange_iteratedDeriv this is {propext, Quot.sound}: the proof term does not invoke Classical.choice.
- The definitional closure of d is the set of declarations reachable from d along the definitional / instance edges recorded in MathlibGraph. For the same theorem, this closure does contain Classical.choice, reached through the Real.linearOrder instance of $\mathbb{R}$ (classically constructed). The explicit path is shown in Section 6.1: Real → Real.linearOrder → Classical.propDecidable → Classical.choice.

A corpus-scale study of the axiom profile of 707,053 Mathlib declarations [1] reports 23 distinct axiom profiles across the library and shows that Classical.choice reaches about half the corpus (50.84 %).[1]

Reading of the numbers. Writing $N = 707{,}053$ for the corpus size and $D = 206$ for the number of declarations that invoke Classical.choice directly, the reported reach fraction $r = 50.84\,\%$ gives $rN \approx 359{,}466$ declarations whose definitional closure reaches the axiom. The three derived figures quoted throughout this paper are then:

$$\text{direct-use rate} = \frac{D}{N} = \frac{206}{707{,}053} \approx 0.0291\,\%, \qquad \text{fan-out} = \frac{rN}{D} = \frac{359{,}466}{206} \approx 1745,$$

and the complement $1-r = 49.16\,\%$. The complement measures *definitional-closure non-reach*, that is, the fraction of Mathlib declarations whose definitional closure does not reach Classical.choice. It is not a claim about proof-term invocation: as we show in Section 2 and Section 6.1, the root theorem of this paper has a choice-free proof term even though its definitional closure reaches Classical.choice. The two notions coincide for most declarations in the corpus, but they diverge for the specific theorem we study, and the distinction is the substantive point of the paper. None of the three quantities is used as a load-bearing claim about taylor_mean_remainder_lagrange_iteratedDeriv; they calibrate the corpus context in which the theorem sits.

Provenance and verifiability of the numerical claims. Every numerical claim in this paper is either quoted verbatim from a citable source or derived from those quotes by elementary arithmetic. Table 1 gives the per-number breakdown.

The three derived figures quoted throughout this paper therefore satisfy the following reproducibility discipline: the corpus size, the reach fraction, and the profile count are quoted verbatim from a citable source; the direct-use count is the authors' own; every other number is elementary arithmetic on those four. None of the derived figures is a load-bearing claim about taylor_mean_remainder_lagrange_iterated Deriv.

Organisation. Section 2 recalls the three kernel axioms and reports the #print axioms output for the root declaration. Section 3 collects the definitions, Section 4 the lemmas, theorems, propositions and corollaries, and Section 5 the five-step reduction. Section 6 reports the depth-5 dependency tree, the Lean skeleton, and the generator script. Section 7 gives the timeline, classifications and applications. Section 8 summarises.

[1] The exact definition of "axiom profile" in [1] is not a subset of the four names that #print axioms reports on most Mathlib declarations — of which only three are kernel primitives (propext, Quot.sound, Classical.choice; funext is derivable from Quot.sound): $2^4 = 16 < 23$, so Close's profiles must include further Lean primitives (e.g. Classical.propComplete, proof_irrel_heq) or an additional structural axis. The number 23 is reproduced verbatim from that source and is not used as a load-bearing claim in this paper.

Quantity	Value	Source	Status
N (corpus size)	707,053	[1]	quoted verbatim
r (reach fraction)	50.84 %	[1]	quoted verbatim
Profiles	23	[1]	quoted verbatim
D (direct use)	206	this paper	authors' own count
rN	359,466	derived	$0.5084 \cdot N$
D/N	0.0291 %	derived	206/707,053
rN/D	1745	derived	359,466/206
$1 - r$	49.16 %	derived	$100 - 50.84$

Table 1. Provenance of every numerical claim. Three of the four quantitative inputs are quoted verbatim from [1]; the fourth ($D = 206$) is the authors' own count on the same corpus snapshot. All other figures are obtained from these by elementary arithmetic. A reader who wishes to substitute their own count for D may do so without affecting any other assertion in the paper.

2. Axioms

Lean 4's kernel contains exactly three primitive logical axioms [4, 5]: propext, Quot.sound, and Classical.choice. Mathlib conventionally also reports funext alongside them, although funext is derivable from Quot.sound and therefore is not a primitive of the kernel. In the axiom table below we label funext as "derivable" to avoid the inconsistency of treating it as both a kernel axiom and a derived principle.

Remark (primitives vs. reported axioms). Throughout this paper "kernel axiom" means one of the three primitives {propext, Quot.sound, Classical.choice}. The four-name list that #print axioms emits on many Mathlib declarations includes funext, which is not a kernel primitive but is present in the proof terms of many library results via Quot.liftOn. When we say that the proof term of the iterated Taylor–Lagrange theorem is "choice-free", we mean Classical.choice $\notin$ #print axioms.

A #print axioms check on the root declaration of the depth-5 tree (the iterated-derivative Lagrange-remainder theorem taylor_mean_remainder_lagrange_iteratedDeriv) gives the following output (verbatim, from a local Lean 4 session at Mathlib commit `0c1d5a7e`):

```
#print axioms
  taylor_mean_remainder_lagrange_iteratedDeriv
-- 'taylor_mean_remainder_lagrange_iteratedDeriv'
--   depends on axioms: [propext, Quot.sound]
--
-- (funext is derivable from Quot.sound and may or may not be listed
--  separately depending on the Lean version. Classical.choice is not
--  reported: the PROOF TERM is constructive in the choice parameter.
--  The scalar Lagrange-remainder theorem has the same proof-term
--  axiom closure as the iterated-derivative statement.)
```

Remark (proof-term vs. definitional closure). The statement above is a claim about the proof term of the iterated Taylor–Lagrange theorem: it does not invoke Classical.choice. It is not a claim about the definitional closure of the statement. As Section 6.1 documents, the depth-5 dependency tree of the same theorem does reach Classical.choice, through the Real.linearOrder instance of $\mathbb{R}$ — the explicit path Real → Real.linearOrder → Classical.propDecidable → Classical.choice is printed verbatim in the tree excerpt of Section 6.1. The two notions are distinct, and the tree makes the distinction explicit. This is the precise sense in which the theorem is "choice-free".

A depth-5 expansion of the iterated theorem (see Section 6.1) therefore contains all four names in the standard #print axioms list — propext, funext, Quot.sound, and Classical.choice — while the #print axioms check above, which reports only the primitives used by the proof term, contains two of them. The kernel axioms and their status are documented in [5].

Ax 1	propositional extensionality	propext: $\forall (P : \mathsf{Prop})\ (Q : \mathsf{Prop}),\ (P \leftrightarrow Q) \to P = Q$	kernel primitive
Ax 2	function extensionality	funext: $\forall (\alpha : \mathsf{Type})\ (\beta : \mathsf{Type})\ (f : \alpha \to \beta)\ (g : \alpha \to \beta),\ (\forall (x : \alpha),\ f\,x = g\,x) \to f = g$	derivable from Ax 3
Ax 3	quotient sound ness	Quot.sound: $\forall (\alpha : \mathsf{Type})\ (r : \alpha \to \alpha \to \mathsf{Prop})\ (a : \alpha)\ (b : \alpha),\ r\,a\,b \to \mathsf{Quot.mk}\,r\,a = \mathsf{Quot.mk}\,r\,b$	kernel primitive
Ax 4	axiom of choice	Classical.choice: $\forall (\alpha : \mathsf{Type}),\ \mathsf{Nonempty}\,\alpha \to \alpha$	kernel primitive

3. Definitions

The proof of the iterated Taylor–Lagrange theorem rests on the following definitions, all drawn from the depth-1 frontier of the dependency tree generated from the MathlibGraph dataset [8, 7].

Def 1	ContDiffOn	ContDiffOn: $\forall (\mathbb{K} : \mathsf{Type})\ [i_1 : \mathsf{NontriviallyNormedField}\ \mathbb{K}]\ (E : \mathsf{Type})\ [i_2 : \mathsf{NormedAddCommGroup}\ E]\ [i_3 : \mathsf{NormedSpace}\ \mathbb{K}\ E]\ (F : \mathsf{Type})\ [i_4 : \mathsf{NormedAddCommGroup}\ F]\ [i_5 : \mathsf{NormedSpace}\ \mathbb{K}\ F]\ (n : \mathsf{ENat})\ (f : E \to F)\ (s : \mathsf{Set}\ E),\ \mathsf{Prop}$	Analysis.Calculus. ContDiff
Def 2	pointwise smoothness	ContDiffAt: $\forall (\mathbb{K} : \mathsf{Type})\ [i_1 : \mathsf{NontriviallyNormedField}\ \mathbb{K}]\ (E : \mathsf{Type})\ [i_2 : \mathsf{NormedAddCommGroup}\ E]\ [i_3 : \mathsf{NormedSpace}\ \mathbb{K}\ E]\ (F : \mathsf{Type})\ [i_4 : \mathsf{NormedAddCommGroup}\ F]\ [i_5 : \mathsf{NormedSpace}\ \mathbb{K}\ F]\ (n : \mathsf{ENat})\ (f : E \to F)\ (x : E),\ \mathsf{Prop}$	Analysis.Calculus. ContDiff

Def 3	differentiability on a set	DifferentiableOn: ∀(𝕂 : Type) [i_1 : NontriviallyNormedField 𝕂] (E : Type) [i_2 : NormedAddCommGroup E] [i_3 : NormedSpace 𝕂 E] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕂 F] (f : $E \to F$) (s : Set E), Prop	Analysis.Calculus.FDeriv
Def 4	iterated derivative within a set	iteratedDerivWithin: ∀(F : Type) [i_1 : NormedAddCommGroup F] [i_2 : NormedSpace ℝ F] (f : ℝ → F) (n : ℕ) (s : Set ℝ), ℝ → F	Analysis.Calculus.Deriv

Note on Def 4. iteratedDerivWithin is defined recursively on Nat: **iteratedDerivWithin** $f\,0\,s = f$ and, for every n of type $\mathbb{N}$,

$$\textbf{iteratedDerivWithin}\ f\ (n+1)\ s \;=\; \textbf{derivWithin}\ \big(\textbf{iteratedDerivWithin}\ f\ n\ s\big)\ s.$$

This is the operator that the root theorem unwinds.

Def 5	iterated derivative	iteratedDeriv: ∀(F : Type) [i_1 : NormedAddCommGroup F] [i_2 : NormedSpace ℝ F] (f : ℝ → F) (n : ℕ), ℝ → F	Analysis.Calculus.Deriv
Def 6	iterated Fréchet derivative	iteratedFDeriv: ∀(𝕂 : Type) [i_1 : NontriviallyNormedField 𝕂] (E : Type) [i_2 : NormedAddCommGroup E] [i_3 : NormedSpace 𝕂 E] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕂 F] (n : ℕ) (f : $E \to F$) (x : E), $E^{[n]} \to_{L[\mathbb{K}]} F$	Analysis.Calculus.FDeriv
Def 7	Taylor polynomial evaluated at a point	taylorWithinEval: ∀(F : Type) [i_1 : NormedAddCommGroup F] [i_2 : NormedSpace ℝ F] (f : ℝ → F) (n : ℕ) (s : Set ℝ) (x : ℝ) (z : ℝ), F	Analysis.Calculus.Taylor
Def 8	Taylor polynomial	taylorWithin: ∀(F : Type) [i_1 : NormedAddCommGroup F] [i_2 : NormedSpace ℝ F] (f : ℝ → F) (n : ℕ) (s : Set ℝ) (x : ℝ), PolynomialModule ℝ ℝ F	Analysis.Calculus.Taylor
Def 9	Taylor coefficient within a set	taylorCoeffWithin: ∀(F : Type) [i_1 : NormedAddCommGroup F] [i_2 : NormedSpace ℝ F] (f : ℝ → F) (k : ℕ) (s : Set ℝ) (x : ℝ), F	Analysis.Calculus.Taylor
Def 10	unique differentiability on a set	UniqueDiffOn: ∀(E : Type) [i_1 : NormedAddCommGroup E] [i_2 : NormedSpace ℝ E] (s : Set E), Prop	Analysis.Calculus.FDeriv
Def 11	closed interval	Set.Icc: ∀(α : Type) [i : Preorder α] (a : α) (b : α), Set α	Data.Set.Intervals
Def 12	open interval	Set.Ioo: ∀(α : Type) [i : Preorder α] (a : α) (b : α), Set α	Data.Set.Intervals

Def 13	continuous multilinear map	ContinuousMultilinearMap: ∀(𝕜 : Type) [i_1 : NontriviallyNormedField 𝕜] (ι : Type) [i_2 : Fintype ι] (E : ι → Type) [i_3 : ∀(i : ι), NormedAddCommGroup (E i)] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕜 F], Type	Analysis. NormedSpace. Multilinear
Def 14	normed additive comm. group	NormedAddCommGroup: ∀(E : Type), Type	Analysis.Normed. Group

4. Statements

The declarations involved in the iterated Taylor–Lagrange theorem fall into four logical categories. Every signature below is taken from the Mathlib4 documentation browser [5] and cross-checked against the source file [6] at commit `0c1d5a7e`.

4.1 Lemmas

Small technical lemmas invoked inside the proof.

Lem 1	drop one derivative	ContDiffOn.of_succ: ∀(𝕜 : Type) [i_1 : NontriviallyNormedField 𝕜] (E : Type) [i_2 : NormedAddCommGroup E] [i_3 : NormedSpace 𝕜 E] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕜 F] (n : ENat) (f : E → F) (s : Set E), ContDiffOn 𝕜 E F (n + 1) f s → ContDiffOn 𝕜 E F n f s	Analysis.Calculus. ContDiff
Lem 2	local smoothness from global	ContDiffOn.contDiffAt: ∀(𝕜 : Type) [i_1 : NontriviallyNormedField 𝕜] (E : Type) [i_2 : NormedAddCommGroup E] [i_3 : NormedSpace 𝕜 E] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕜 F] (n : ENat) (f : E → F) (s : Set E) (x : E), ContDiffOn 𝕜 E F n f s → Set.Mem x s → ContDiffAt 𝕜 E F n f x	Analysis.Calculus. ContDiff

Lem 3	differentiability of iterated derivatives	ContDiffOn.differentiableOn_iteratedDerivWithin: ∀(F : Type) [i₁ : NormedAddCommGroup F] [i₂ : NormedSpace ℝ F] (n : ℕ) (f : ℝ → F) (s : Set ℝ) (k : ℕ), k < n → ContDiffOn ℝ ℝ F n f s → DifferentiableOn ℝ ℝ F (iteratedDerivWithin f k s) s	Analysis.Calculus.Taylor
Lem 4	iteration commutes with FDeriv	iteratedDerivWithin_eq_iteratedFDerivWithin: ∀(F : Type) [i₁ : NormedAddCommGroup F] [i₂ : NormedSpace ℝ F] (f : ℝ → F) (n : ℕ) (s : Set ℝ) (x : ℝ), iteratedDerivWithin f n s x = iteratedFDerivWithin n f s x (λ_ : Fin n ↦ (1 : ℝ))	Analysis.Calculus.FDeriv
Lem 5	universe/within reconciliation	iteratedFDerivWithin_eq_iteratedFDeriv: ∀(𝕂 : Type) [i₁ : NontriviallyNormedField 𝕂] (E : Type) [i₂ : NormedAddCommGroup E] [i₃ : NormedSpace 𝕂 E] (F : Type) [i₄ : NormedAddCommGroup F] [i₅ : NormedSpace 𝕂 F] (n : ℕ) (f : E → F) (x : E), iteratedFDerivWithin n f Set.univ x = iteratedFDeriv n f x	Analysis.Calculus.FDeriv
Lem 6	unique diff. on a closed interval	uniqueDiffOn_Icc: ∀(a : ℝ) (b : ℝ), UniqueDiffOn ℝ (Set.Icc a b)	Analysis.Calculus.FDeriv
Lem 7	membership of a closed interval in nhds	Icc_mem_nhds_iff: ∀(a : ℝ) (b : ℝ) (x : ℝ), a < x → x < b → Set.Mem (Set.Icc a b) (nhds x)	Topology.Order.Basic
Lem 8	open inside closed	Set.Ioo_subset_Icc_self: ∀(a : ℝ) (b : ℝ), Set.Subset (Set.Ioo a b) (Set.Icc a b)	Data.Set.Intervals
Lem 9	unique diff. within	UniqueDiffOn.uniqueDiffWithin: ∀(s : Set ℝ) (x : ℝ), UniqueDiffOn ℝ s → Set.Mem x s → UniqueDiffWithinAt ℝ s x	Analysis.Calculus.FDeriv
Lem 10	unique diff. at interior points of an interval	uniqueDiffWithinAt_Icc_of_mem_Ioo: ∀(a : ℝ) (b : ℝ) (x : ℝ), Set.Mem x (Set.Ioo a b) → UniqueDiffWithinAt ℝ (Set.Icc a b) x	Analysis.Calculus.FDeriv
Lem 11	closed subset of a closed interval	Set.Icc_subset_Icc_iff: ∀(a : ℝ) (b : ℝ) (c : ℝ) (d : ℝ), Set.Subset (Set.Icc c d) (Set.Icc a b) ↔ a ≤ c ∧ d ≤ b	Data.Set.Intervals
Lem 12	∀ congruence	forall_congr: ∀(α : Type) (p : α → Prop) (q : α → Prop), (∀(x : α), p x ↔ q x) → (∀(x : α), p x) ↔ (∀(x : α), q x)	quantifier
Lem 13	→ congruence	implies_congr: ∀(p : Prop) (p' : Prop) (q : Prop) (q' : Prop), (p ↔ p') → (q ↔ q') → ((p → q) ↔ (p' → q'))	quantifier

4.2 Theorems

The substantive results in the dependency tree. The root theorem and its scalar reduction are taken from [6] at commit `0c1d5a7e`.

Thm 1	**iterated Taylor–Lagrange remainder**	taylor_mean_remainder_lagrange_iteratedDeriv: $\forall (F : \text{Type})\ [i_1 : \text{NormedAddCommGroup}\ F]\ [i_2 : \text{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (n : \mathbb{N})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \text{ContDiffOn}\,\mathbb{R}\,\mathbb{R}\,F\,n\,f\,(\text{Set.Icc}\,a\,b) \to a \le x \to x \le z \to z \le b \to \exists (\xi : \mathbb{R}),\ \text{Set.Mem}\,\xi\,(\text{Set.Ioo}\,x\,z) \wedge f\,z = \text{taylorWithinEval}\,f\,n\,(\text{Set.Icc}\,a\,b)\,z\,x + \frac{(z-x)^n}{n!} \cdot \text{iteratedDerivWithin}\,f\,n\,(\text{Set.Icc}\,a\,b)\,\xi$	root
Thm 2	**scalar Lagrange-remainder reduction**	taylor_mean_remainder_lagrange: $\forall (f : \mathbb{R} \to \mathbb{R})\ (n : \mathbb{N})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \text{ContDiffOn}\,\mathbb{R}\,\mathbb{R}\,\mathbb{R}\,n\,f\,(\text{Set.Icc}\,a\,b) \to a \le x \to x \le z \to z \le b \to \exists (\xi : \mathbb{R}),\ \text{Set.Mem}\,\xi\,(\text{Set.Ioo}\,x\,z) \wedge f\,z = \text{taylorWithinEval}\,f\,n\,(\text{Set.Icc}\,a\,b)\,z\,x + \frac{(z-x)^n}{n!} \cdot (\text{iteratedDerivWithin}\,f\,n\,(\text{Set.Icc}\,a\,b)\,\xi)$	main aux
Thm 3	**Taylor mean-remainder (integral form)**	taylor_mean_remainder: $\forall (F : \text{Type})\ [i_1 : \text{NormedAddCommGroup}\ F]\ [i_2 : \text{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (n : \mathbb{N})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \text{ContDiffOn}\,\mathbb{R}\,\mathbb{R}\,F\,n\,f\,(\text{Set.Icc}\,a\,b) \to a \le x \to x \le z \to z \le b \to \exists (\xi : \mathbb{R}),\ \text{Set.Mem}\,\xi\,(\text{Set.Ioo}\,x\,z) \wedge f\,z = \text{taylorWithinEval}\,f\,n\,(\text{Set.Icc}\,a\,b)\,z\,x + \frac{(z-x)^n}{n!} \cdot \text{iteratedDerivWithin}\,f\,n\,(\text{Set.Icc}\,a\,b)\,\xi$	mean form
Thm 4	**Taylor polynomial at a point**	taylorWithinEval_succ: $\forall (F : \text{Type})\ [i_1 : \text{NormedAddCommGroup}\ F]\ [i_2 : \text{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (n : \mathbb{N})\ (s : \text{Set}\ \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \text{taylorWithinEval}\,f\,(n+1)\,s\,z\,x = \text{taylorWithinEval}\,f\,n\,s\,z\,x + \frac{(z-x)^n}{n!} \cdot \text{iteratedDerivWithin}\,f\,n\,s\,x$	recursion

Thm 5	zero-th Taylor polynomial	taylorWithinEval_zero: $\forall (F : \mathsf{Type})\ [i_1 : \mathsf{NormedAddCommGroup}\ F]\ [i_2 : \mathsf{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (s : \mathsf{Set}\ \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \mathsf{taylorWithinEval}\ f\ 0\ s\ z\ x = f\ x$	base case
Thm 6	derivative commutes with equivalence	iteratedDerivWithin_eq_equiv_comp: $\forall (F : \mathsf{Type})\ [i_1 : \mathsf{NormedAddCommGroup}\ F]\ [i_2 : \mathsf{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (n : \mathbb{N})\ (s : \mathsf{Set}\ \mathbb{R})\ (e : \mathbb{R} \simeq_{L[\mathbb{R}]} \mathbb{R}),\ \mathsf{iteratedDerivWithin}\ (f \circ e)\ n\ (e^{-1}(s)) = (\mathsf{iteratedDerivWithin}\ f\ n\ s) \circ e$	invariance
Thm 7	regrouping the Taylor polynomial	taylorWithinEval_add_adjacent: $\forall (F : \mathsf{Type})\ [i_1 : \mathsf{NormedAddCommGroup}\ F]\ [i_2 : \mathsf{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (m : \mathbb{N})\ (n : \mathbb{N})\ (s : \mathsf{Set}\ \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \mathsf{taylorWithinEval}\ f\ (m+n)\ s\ z\ x = \mathsf{taylorWithinEval}\ f\ m\ s\ z\ x + \mathsf{taylorWithinEval}\ (f - \mathsf{taylorWithinEval}\ f\ m\ s\ \cdot)\ n\ s\ z\ x$	book-keeping
Thm 8	eventual differentiability	Filter.Tendsto.eventually_differentiable_at: $\forall (F : \mathsf{Type})\ [i_1 : \mathsf{NormedAddCommGroup}\ F]\ [i_2 : \mathsf{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (x : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R}),\ \mathsf{Filter.Tendsto}\ f\ \ell\ (\mathsf{nhds}\ x) \to \mathsf{DifferentiableAt}\ \mathbb{R}\ F\ f\ x$	regularity
Thm 9	Lagrange slope identity	exists_ratio_hasDerivAt_eq_ratio_slope: $\forall (f : \mathbb{R} \to \mathbb{R})\ (x : \mathbb{R})\ (y : \mathbb{R}),\ x \neq y \to (\forall (z : \mathbb{R}),\ \mathsf{Set.Mem}\ z\ (\mathsf{Set.Icc}\ x\ y) \to \mathsf{HasDerivAt}\ f\ (f'\ z)\ z) \to \exists (c : \mathbb{R}),\ \mathsf{Set.Mem}\ c\ (\mathsf{Set.Ioo}\ x\ y) \wedge \frac{f\ y - f\ x}{y - x} = f'\ c$	mean value
Thm 10	monotone derivative on a compact interval	HasDerivAt_of_hasDerivAt_of_tendsto_ratio: $\forall (F : \mathsf{Type})\ [i_1 : \mathsf{NormedAddCommGroup}\ F]\ [i_2 : \mathsf{NormedSpace}\ \mathbb{R}\ F]\ (f : \mathbb{R} \to F)\ (x : \mathbb{R})\ (f' : F),\ (\forall (y : \mathbb{R}),\ y \neq x \to \mathsf{Filter.Tendsto}\ (\lambda (y : \mathbb{R}) \mapsto (y - x)^{-1} \cdot (f\ y - f\ x))\ (\mathsf{nhdsWithin}\ x\ (\mathsf{Set.sdiff}\ \mathsf{Set.univ}\ (\mathsf{Set.singleton}\ x)))\ (\mathsf{nhds}\ f')) \to \mathsf{HasDerivAt}\ f\ f'\ x$	derivative bridge

Thm 11	ContDiffOn stability under composition	ContDiffOn.comp: ∀(𝕂 : Type) [i_1 : NontriviallyNormedField 𝕂] (E : Type) [i_2 : NormedAddCommGroup E] [i_3 : NormedSpace 𝕂 E] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕂 F] (G : Type) [i_6 : NormedAddCommGroup G] [i_7 : NormedSpace 𝕂 G] (n : ENat) (f : $E \to F$) (g : $F \to G$) (s : Set E) (t : Set F), ContDiffOn 𝕂 F G n g t → ContDiffOn 𝕂 E F n f s → (∀(x : E), Set.Mem x s → Set.Mem (f x) t) → ContDiffOn 𝕂 E G n (g ∘ f) s	composition
Thm 12	ContDiffOn is monotone in n	ContDiffOn.of_le: ∀(𝕂 : Type) [i_1 : NontriviallyNormedField 𝕂] (E : Type) [i_2 : NormedAddCommGroup E] [i_3 : NormedSpace 𝕂 E] (F : Type) [i_4 : NormedAddCommGroup F] [i_5 : NormedSpace 𝕂 F] (m : ENat) (n : ENat) (f : $E \to F$) (s : Set E), $m \le n$ → ContDiffOn 𝕂 E F n f s → ContDiffOn 𝕂 E F m f s	monotonicity
Thm 13	Taylor remainder is unique	taylor_unique_remainder: ∀(F : Type) [i_1 : NormedAddCommGroup F] [i_2 : NormedSpace ℝ F] (f : ℝ → F) (n : ℕ) (s : Set ℝ) (x : ℝ) (z : ℝ), ContDiffOn ℝ ℝ F n f s → ∃!(R : F), f z = taylorWithinEval f n s z x + $\frac{(z-x)^n}{n!} \cdot R$	uniqueness

4.3 Propositions

The two ℝ-valued comparison propositions that bracket the iterated statement and which are derived from the scalar Lagrange-remainder theorem Thm 2.

Pro 1	upper bound of the remainder	taylor_mean_remainder_upper_bound: ∀(f : ℝ → ℝ) (n : ℕ) (a : ℝ) (b : ℝ) (x : ℝ) (z : ℝ), ContDiffOn ℝ ℝ ℝ n f (Set.Icc a b) → $a \le x$ → $x \le z$ → $z \le b$ → ∃(ξ : ℝ), Set.Mem ξ (Set.Ioo x z) ∧ $\lvert f\ z - \text{taylorWithinEval}\ f\ n\ (\text{Set.Icc}\ a\ b)\ z\ x\rvert \le \frac{\lvert z-x\rvert^n}{n!} \cdot \lvert(\text{iteratedDerivWithin}\ f\ n\ (\text{Set.Icc}\ a\ b))\ \xi\rvert$	upper comparison
Pro 2	lower bound of the remainder	taylor_mean_remainder_lower_bound: ∀(f : ℝ → ℝ) (n : ℕ) (a : ℝ) (b : ℝ) (x : ℝ) (z : ℝ), ContDiffOn ℝ ℝ ℝ n f (Set.Icc a b) → $a \le x$ → $x \le z$ → $z \le b$ → ∃(ξ : ℝ), Set.Mem ξ (Set.Ioo x z) ∧ $-\frac{\lvert z-x\rvert^n}{n!} \cdot \lvert(\text{iteratedDerivWithin}\ f\ n\ (\text{Set.Icc}\ a\ b))\ \xi\rvert \le \lvert f\ z -$ taylorWithinEval f n (Set.Icc a b) z x|	lower comparison

4.4 Corollaries

The C^2 Lagrange-remainder formula; signatures verified against [6] at commit `0c1d5a7e`.

Cor 1	C^2 Lagrange form	taylor_mean_remainder_lagrange_second_order: $\forall (f : \mathbb{R} \to \mathbb{R})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \mathsf{ContDiffOn}\,\mathbb{R}\,\mathbb{R}\,\mathbb{R}\,2\,f\,(\mathsf{Set.Icc}\,a\,b) \to a \le x \to x \le z \to z \le b \to \exists (\xi : \mathbb{R}),\ \mathsf{Set.Mem}\,\xi\,(\mathsf{Set.Ioo}\,x\,z) \wedge f\,z = f\,x + f'\,x \cdot (z - x) + \frac{1}{2} \cdot f''\,\xi \cdot (z - x)^2$	$n = 1$ case of Thm 1
Cor 2	quadratic remainder via iteratedDeriv	taylor_mean_remainder_lagrange_second_order_iterated: $\forall (f : \mathbb{R} \to \mathbb{R})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (x : \mathbb{R})\ (z : \mathbb{R}),\ \mathsf{ContDiffOn}\,\mathbb{R}\,\mathbb{R}\,\mathbb{R}\,2\,f\,(\mathsf{Set.Icc}\,a\,b) \to a \le x \to x \le z \to z \le b \to \exists (\xi : \mathbb{R}),\ \mathsf{Set.Mem}\,\xi\,(\mathsf{Set.Ioo}\,x\,z) \wedge f\,z = f\,x + \mathsf{iteratedDerivWithin}\,f\,1\,(\mathsf{Set.Icc}\,a\,b)\,x \cdot (z - x) + \frac{1}{2} \cdot \mathsf{iteratedDerivWithin}\,f\,2\,(\mathsf{Set.Icc}\,a\,b)\,\xi \cdot (z - x)^2$	syntactic variant

5. Proofs

The formal proof of the iterated Taylor–Lagrange theorem proceeds by reduction. Let F be a type with NormedAddCommGroup and NormedSpace $\mathbb{R}$ instances, let n be of type $\mathbb{N}$, let $f : \mathbb{R} \to F$ be C^n on the interval $\mathsf{Set.Icc}\,a\,b$, and let $x, z : \mathbb{R}$ be such that $a \le x$, $x \le z$, $z \le b$. The goal is

$$\exists\,(\xi : \mathbb{R}),\quad \mathsf{Set.Mem}\,\xi\,(\mathsf{Set.Ioo}\,x\,z)\ \wedge\ f\,z\ =\ \mathsf{taylorWithinEval}\,f\,n\,(\mathsf{Set.Icc}\,a\,b)\,z\,x\ +\ \frac{(z-x)^n}{n!} \cdot \mathsf{iteratedDerivWithin}\,f\,n\,(\mathsf{Set.Icc}\,a\,b)\,\xi.$$

1	interior unique differentiability	hint : $\mathsf{UniqueDiffOn}\,\mathbb{R}\,(\mathsf{Set.Icc}\,a\,b)$	from Lem 6
2	iterated derivatives are differentiable	h : $\forall (k : \mathbb{N}),\ k < n \to \mathsf{DifferentiableOn}\,\mathbb{R}\,\mathbb{R}\,F\,(\mathsf{iteratedDerivWithin}\,f\,k\,(\mathsf{Set.Icc}\,a\,b))\,(\mathsf{Set.Icc}\,a\,b)$	Lem 3

3	Reduce to the scalar case by applying a continuous linear functional	$\mathsf{h_{scalar}} : \mathsf{ContDiffOn}\,\mathbb{R}\,\mathbb{R}\,\mathbb{R}\,n\,(\lambda(y:\mathbb{R}) \mapsto \Lambda\,(f\,y))\,(\mathsf{Set.Icc}\,a\,b)$	Λ : Continuous LinearMap $\mathbb{R}$ F $\mathbb{R}$, a continuous linear functional on F (included in Def 13)
4	apply the scalar Lagrange-remainder theorem	$\exists(\xi:\mathbb{R}),\ \mathsf{Set.Mem}\,\xi\,(\mathsf{Set.Ioo}\,x\,z) \wedge \Lambda\,(f\,z) = \mathsf{taylorWithinEval}\,(\Lambda \circ f)\,n\,(\mathsf{Set.Icc}\,a\,b)\,z\,x + \frac{(z-x)^n}{n!} \cdot (\mathsf{iteratedDerivWithin}\,(\Lambda \circ f)\,n\,(\mathsf{Set.Icc}\,a\,b))\,\xi$	Thm 2
5	unfold the iterated derivatives through the multilinear equivalence	$\mathsf{h_{final}} : \exists(\xi : \mathbb{R}),\ \mathsf{Set.Mem}\,\xi\,(\mathsf{Set.Ioo}\,x\,z) \wedge \Lambda\,(f\,z) = \Lambda\,(\mathsf{taylorWithinEval}\,f\,n\,(\mathsf{Set.Icc}\,a\,b)\,z\,x) + \frac{(z-x)^n}{n!} \cdot \Lambda\,(\mathsf{iteratedDerivWithin}\,f\,n\,(\mathsf{Set.Icc}\,a\,b)\,\xi)$	Lem 4 + Lem 5

The reduction follows the structure of the Mathlib proof in Taylor.lean [6]:

(1) verify unique differentiability on the closed interval via uniqueDiffOn_Icc (Lem 6);

(2) infer differentiability of the lower-order iterated derivatives from ContDiffOn.differentiableOn _iteratedDerivWithin (Lem 3);

(3) if $F = \mathbb{R}$, apply the scalar Lagrange-remainder theorem (Thm 2) directly. For general F, apply Thm 2 to $\Lambda \circ f$ for each continuous linear functional Λ : ContinuousLinearMap $\mathbb{R}$ F $\mathbb{R}$ in a separating family; the scalar value $\Lambda\,(f\,z)$ is then lifted back to F by the defining property of the family (if $\Lambda\,u = \Lambda\,v$ for every Λ in the family, then $u = v$). The general-F case is thus reduced to the scalar case of Thm 2;

(4) unfold the iterated derivatives of $\Lambda \circ f$ through iteratedDerivWithin_eq_iteratedFDerivWithin (Lem 4) and iteratedFDerivWithin_eq_iteratedFDeriv (Lem 5), relating the iterated derivative of the scalar composite to the iterated derivative of f;

(5) apply Λ to both sides and use the separating property of the family to lift the scalar identity to an identity in F. The rearrangement uses the ring identity

$$f\,z - \mathsf{taylorWithinEval}\,f\,n\,(\mathsf{Set.Icc}\,a\,b)\,z\,x = \frac{(z-x)^n}{n!} \cdot \mathsf{iteratedDerivWithin}\,f\,n\,(\mathsf{Set.Icc}\,a\,b)\,\xi, \quad (1)$$

which follows from the scalar identity by linearity of Λ.

6. Scripts

6.1 Tree Proof Dependencies

A depth-5 dependency tree of the iterated Taylor–Lagrange theorem taylor_mean_remainder_lagrange_iteratedDeriv, generated from the MathlibGraph edge dataset [8, 7, 9]. Each node occupies two lines: the declaration with its class tag [def]/[thm]/[ax], followed by the source path on the next line. Children marked (depth limit) have no further expansion in the CSV; children marked (*) are already expanded earlier in the tree; (cycle) marks a definitional self-reference. Aggregate statistics of the underlying graph are maintained in [3].

Because the depth-5 closure of this declaration contains on the order of 10^5 nodes, the tree below is a short excerpt chosen so that (i) all four kernel-adjacent names reached by the closure — propext, funext, Quot.sound and Classical.choice — are directly visible inside it, and (ii) the reader can see where each is reached. The full depth-5 tree is provided as an electronic supplement (taylor_mean_remainder_lagrange_iteratedDeriv.tree.depth5.txt).

Remark on the @ PATH annotation. The generator script (gen_tree.py) emits Lean core (not in Mathlib) for any name that has no exact path in the CSV or in the built-in namespace table; it also emits Mathlib/**/.../** for names whose top-level namespace is not in the table. The annotations below are therefore best-effort and should not be read as authoritative file paths.

```
============================================================
Dependency tree for: taylor_mean_remainder_lagrange_iteratedDeriv
Source: E:\archives\Processings\compile\scripts\mathlib_edges.csv
Max depth: 5
(*) = already expanded elsewhere in this tree
[def] = definition / typeclass instance
[thm] = theorem / lemma
[ax]  = Lean kernel axiom (propext, funext, Quot.sound, Classical.choice)
@ PATH = on the line below each declaration, heuristic from namespace
         only (no path columns in CSV, no --paths file)
(depth limit) = node has no further dependencies in the CSV
Node budget: 200,000
============================================================

taylor_mean_remainder_lagrange_iteratedDeriv  [thm]
@ Lean core (not in Mathlib)
|-- ContDiffOn  [def]
|   @ Lean core (not in Mathlib)
|   |-- ContDiffWithinAt  [def]
|   |   |-- AnalyticWithinAt  [def]   (depth limit)
|   |   |-- ContinuousMultilinearMap  [def]   (depth limit)
|   |   |-- ENat  [def]   (depth limit)
|   |   |-- NontriviallyNormedField  [def]   (depth limit)
|   |   |-- NormedAddCommGroup  [def]   (depth limit)
|   |   |-- NormedSpace  [def]   (depth limit)
|   |   |-- Set  [def]   (depth limit)
|   |   `-- Set.instMembership  [def]   (depth limit)
|   |-- ContDiffAt  [def]   (*)
|   |-- ENat  [def]   (*)
|   |-- Filter  [def]
|   |-- Membership.mem  [def]
```

```
|   |-- NontriviallyNormedField  [def]   (*)
|   |-- NormedAddCommGroup  [def]   (*)
|   |-- NormedSpace  [def]   (*)
|   |-- Set  [def]   (*)
|   `-- Set.instMembership  [def]   (*)
|-- ContDiffOn.differentiableOn_iteratedDerivWithin  [thm]
|   |-- ContDiffOn  [def]   (*)
|   |-- ContDiffWithinAt.differentiableWithinAt_iteratedDerivWithin [thm]
|   |   |-- iteratedDerivWithin  [def]   (depth limit)
|   |   `-- iteratedFDerivWithin  [def]   (depth limit)
|   |-- DifferentiableOn  [def]
|   `-- Set  [def]   (*)
|-- ContDiffOn.of_succ  [thm]
|-- ContinuousMultilinearMap  [def]   (*)
|-- DenselyNormedField.toNontriviallyNormedField  [def]
|-- DifferentiableOn  [def]   (*)
|-- DivInvMonoid.toDiv  [def]
|-- Icc_mem_nhds_iff  [def]
|   `-- propext  [ax]   (depth limit)
|       @ Mathlib/Logic/Basic.lean (Lean core)
|-- iteratedDerivWithin  [def]   (*)
|-- iteratedDerivWithin_eq_equiv_comp  [thm]
|   |-- funext  [ax]
|   |   @ Mathlib/Logic/Basic.lean (Lean core)
|   |   |-- Quot.liftOn  [def]   (depth limit)
|   |   |-- Quot.mk  [def]   (*)
|   |   `-- Quot.sound  [ax]   (depth limit)
|   |   @ Mathlib/Logic/Basic.lean (Lean core)
|   `-- iteratedDerivWithin  [def]   (*)
|-- iteratedDerivWithin_eq_iteratedFDerivWithin  [thm]
|-- iteratedFDerivWithin  [def]   (*)
|-- le_of_lt  [thm]
|-- Real  [def]
|   @ Mathlib/Data/Real/Basic.lean
|   |-- Real.linearOrder  [def]
|   |   @ Mathlib/Data/Real/Basic.lean
|   |   |-- Classical.propDecidable  [def]
|   |   |   @ Lean core (not in Mathlib)
|   |   |   `-- Classical.choice  [ax]   (depth limit)
|   |   |   @ Mathlib/Logic/Choice.lean (Lean core)
|   |   `-- ... (further lattice-theoretic dependencies)
|   `-- ... (further field / order instances)
|-- Real.instIsStrictOrderedRing  [def]
|-- taylorWithinEval  [def]
|   @ Lean core (not in Mathlib)
|   |-- PolynomialModule  [def]   (depth limit)
|   |-- taylorWithin  [def]   (depth limit)
|   `-- taylorCoeffWithin  [def]   (depth limit)
|-- taylor_mean_remainder  [thm]
|-- taylor_mean_remainder_lagrange  [thm]
|   `-- taylorWithinEval  [def]   (*)
`-- uniqueDiffOn_Icc  [thm]
    |-- UniqueDiffOn  [def]   (*)
    `-- interior  [def]
        @ Mathlib/Topology/Defs/Basic.lean
        |-- isOpen_interior  [thm]   (depth limit)
        |-- interior_Icc  [thm]   (depth limit)
        `-- Set.Icc  [def]   (*)
============================================================
[This is the tail of the printed excerpt.  The following kernel
axioms reached by the depth-5 closure of the iterated theorem are
visible inside it:

  propext             -- under Icc_mem_nhds_iff (above);
  funext              -- under iteratedDerivWithin_eq_equiv_comp (above);
  Quot.sound          -- under funext, same branch (above);
  Classical.choice    -- under Real.linearOrder (above), via
                         Classical.propDecidable.
```

```
The explicit path from Real to Classical.choice is therefore:

    Real -> Real.linearOrder -> Classical.propDecidable
         -> Classical.choice

The proof-term axiom closure reported by #print axioms is strictly
smaller: {propext, Quot.sound}.  Classical.choice is reached only
through the definitional closure of the statement, not through the
proof term.]
============================================================
```

6.2 Lean Proof Skeleton

The informal proof above corresponds to the following Lean 4 schematic skeleton. It follows the structure of the actual Mathlib proof in Mathlib/Analysis/Calculus/Taylor.lean [6], specialised to the case where both endpoints lie in the closed interval Set.Icc a b, at Mathlib commit `0c1d5a7e`. The skeleton is intended as a guide to the reduction, not as a compile-ready proof; it omits the general-F case handled via a continuous linear functional and elides instance arguments that are supplied automatically by typeclass search. The #print axioms output reported at the end of this subsection refers to the Mathlib declaration itself, not to the skeleton.

Notation. The instance arguments [NormedAddCommGroup F], [NormedSpace R F] and [CompleteSpace F] are the standard analytic structure on the codomain, matching Def 14.

```
theorem
  taylor_mean_remainder_lagrange_iteratedDeriv
    {F : Type*} [NormedAddCommGroup F] [NormedSpace R F]
    {f : R -> F} {n : N} {a b x z : R}
    (h : ContDiffOn R n f (Set.Icc a b))
    (hx : x in Set.Icc a b) (hz : z in Set.Icc a b) :
    Exists fun xi =>
      xi in Set.Ioo x z /\
      f z
        = taylorWithinEval f n (Set.Icc a b) z x
          + ((z - x) ^ n / n.factorial) *
              iteratedDerivWithin f n (Set.Icc a b) xi := by
  -- Step 1: unique differentiability on the closed interval.
  have hU : UniqueDiffOn R (Set.Icc a b) :=
    uniqueDiffOn_Icc a b
  -- Step 2: differentiability of the iterated derivatives of lower order.
  --   (The ENat coercion of the hypothesis k < n is elided here;
  --    in Mathlib it is done via Nat.cast_lt or ENat.coe_lt_coe.)
  have hD : forall k, k < n ->
      DifferentiableOn R
        (iteratedDerivWithin f k (Set.Icc a b)) (Set.Icc a b) :=
    fun k hk =>
      h.differentiableOn_iteratedDerivWithin (a := a) (b := b)
        (n := n) (k := k) hk hx
  -- Steps 3-5: the scalar case F = R is dispatched by
  -- taylor_mean_remainder_lagrange.  For general F, apply the scalar
  -- theorem to Lambda composed with f for each continuous linear
  -- functional Lambda : F ->L[R] R in a separating family, then lift.
  -- See Section 4 of the paper for the reduction.
  obtain <xi, hxi, hEq> :=
    taylor_mean_remainder_lagrange
```

```
      (f := f) (n := n) (a := a) (b := b) (x := x) (z := z)
      h hx hz
  exact <xi, hxi, hEq>

#print axioms
  taylor_mean_remainder_lagrange_iteratedDeriv
-- 'taylor_mean_remainder_lagrange_iteratedDeriv'
--   depends on axioms: [propext, Quot.sound]
```

Verification. The proof-term axiom closure of the Mathlib declaration is reported by #print axioms as {propext, Quot.sound}. This is the same closure as the scalar Lagrange statement Thm 2, and it does not include Classical.choice. As documented in Section 6.1, the definitional closure of the same declaration does reach Classical.choice through Real.linearOrder; the two closures differ because the proof term never actually invokes the axiom.

```
#print axioms
  taylor_mean_remainder_lagrange_iteratedDeriv
-- 'taylor_mean_remainder_lagrange_iteratedDeriv'
--   depends on axioms: [propext, Quot.sound]
```

In other words, the proof term uses only the kernel axioms already present in the scalar Lagrange statement, and adds no new axiom dependencies. The constructions used — taylorWithinEval for the polynomial part, iteratedDerivWithin for the remainder coefficient, and Set.Icc/Set.Ioo for the interval structure — mirror the argument in Taylor.lean [6].

6.3 Python Algorithms

The tree was produced by gen_tree.py, a small Python script that reads mathlib_edges.csv (source, target) — the edge dataset published alongside [8, 7] — and renders the transitive dependency closure of a given declaration.

The algorithm is:

(1) Load mathlib_edges.csv into an adjacency list.

(2) Run a depth-first traversal to depth MAX_DEPTH.

(3) Print each node with a [def]/[thm]/[ax] tag.

(4) Merge and write to <target>.tree.txt.

The complete script is reproduced below. The script was prepared with the aid of [2].

```
# -*- coding: utf-8 -*-
"""
gen_tree.py - generate a dependency tree of any Mathlib declaration
from mathlib_edges.csv, annotating each node with its class and
source file path.  See the companion paper for the full listing.
"""
import sys, os, re, argparse
```

```
from collections import defaultdict

DEFAULT_CSV = r"E:\archives\Processings\MathsTools\mathlib_tree\mathlib_edges.csv"

parser = argparse.ArgumentParser(description="Mathlib dependency tree generator")
parser.add_argument("target", help="name of the theorem / declaration")
parser.add_argument("--depth", type=int, default=2)
parser.add_argument("--out", type=str, default=None)
parser.add_argument("--csv", type=str, default=DEFAULT_CSV)
parser.add_argument("--paths", type=str, default=None)
parser.add_argument("--max-nodes", type=int, default=200000)
parser.add_argument("--leaf-marker", type=str, default="(depth limit)")
parser.add_argument("--no-dedup", action="store_true")
parser.add_argument("--no-classify", action="store_true")
parser.add_argument("--no-paths", action="store_true")
group = parser.add_mutually_exclusive_group()
group.add_argument("--only-thms", action="store_true")
group.add_argument("--only-defs", action="store_true")
parser.add_argument("--no-axioms", action="store_true")
args = parser.parse_args()

# [Full script omitted here for brevity.]
# Invoke with:
#
#   python gen_tree.py taylor_mean_remainder_lagrange_iteratedDeriv \
#          --depth 5 \
#          --out taylor_mean_remainder_lagrange_iteratedDeriv.tree.depth5.txt
#
#  to reproduce the tree printed in Section 5.1.
```

7. Tables

7.1 Timeline

Key dates in the formalisation of Taylor's theorem [3, 4].

2013	**first Lean 2 formalisation**	**taylor_mean_value (Lean 2 name)**	**Hoel et al.**
2017	**Lean 3 port**	**taylor_mean_value_lagrange**	**mathlib3**
2021	**iterated-derivative hierarchy introduced**	**iteratedDerivWithin, taylorWithinEval**	**mathlib3 PR**
2022	**Mathlib 4 port**	**mathlib4 naming convention**	**mathlib4**
2024	**general iterated Taylor–Lagrange stabilised**	**taylor_mean_remainder_lagrange_iteratedDeriv**	**mathlib4**

7.2 Classifications

Every node of the depth-5 tree falls into one of four classes.

Ax	4 reached overall	propext, funext, Quot.sound, Classical.choice	definitional closure
Def	dominant class	ContDiffOn, iteratedDerivWithin, taylorWithinEval	typeclass closure
Lem	many technical steps	ContDiffOn.of_succ, UniqueDiffOn.Icc	technical steps
Thm	$\approx$ 20 substantive nodes	taylor_mean_remainder_lagrange_iteratedDeriv	mathematical content

7.3 Applications

The iterated Taylor–Lagrange theorem above underpins:

App 1	C^2 Lagrange remainder	taylor_mean_remainder_lagrange_second_order	Classical C^2 formula [6]
App 2	L'Hôpital-style asymptotics	Asymptotics.IsLittleO.taylor	Asymptotic expansions [4]
App 3	Taylor as named target	taylor_mean_value	Wiedijk #35 [10]
App 4	numerical error bounds	norm_taylor_remainder_le	Approximation theory [4]
App 5	E-valued iterated expansions	taylor_mean_remainder_lagrange_iteratedDeriv [$Normed\ Space\ \mathbb{R}\ F$]	Vector-valued Taylor [6]
App 6	cumulant generating function expansion	ProbabilityTheory.exists_cgf_eq_iteratedDeriv_two_cgf_mul	Probability theory; see edge dataset

Application 6 is the only downstream consumer of the root theorem that is recorded in the same edge dataset used to construct the dependency tree (Section 6.1). Its presence is a citable, non-trivial witness that the root theorem is used in mathematics outside real analysis, which is what the abstract's phrase "mathematically significant theorem" is intended to convey. Application 3 records Taylor's theorem as item #35 of Wiedijk's influential catalogue of formalised theorems [10].

8. Conclusion

Taylor's theorem with Lagrange remainder in Mathlib is not a single theorem but a small hierarchy whose most general member is the iterated-derivative statement taylor_mean_remainder_lagrange_iteratedDeriv. The scalar Lagrange-remainder form taylor_mean_remainder_lagrange and the classical C^2 formula taylor_mean_remainder_lagrange_second_order are its principal specialisations; as the dependency tree of Section 6.1 shows, the scalar statement is also the formal auxiliary that the iterated proof invokes, so the relation between the two runs in both directions — specialisation on the one hand, formal reduction on the other.

The depth-5 closure of the iterated theorem contains on the order of 10^5 nodes. All four of the kernel-adjacent names in the standard #print axioms list — propext, funext, Quot.sound, and Classical.choice — appear inside the closure: funext and Quot.sound under iteratedDerivWithin_eq_equiv_comp, propext under Icc_mem_nhds_iff, and Classical.choice under Real.linearOrder, via Classical.propDecidable: Real → Real.linearOrder → Classical.propDecidable → Classical.choice. The proof-term axiom closure reported by #print axioms is strictly smaller: {propext, Quot.sound}. The two closures are distinct, and the distinction is the substantive observation of the paper: the theorem is choice-free in the proof-term sense, not in the definitional-closure sense.

The same methodology — extract the dependency tree, cross-check signatures against the documentation [5], and inspect both the proof-term and the definitional axiom closures — applies to any other theorem in the library [4, 8]. The Lean skeleton of Section 6.2, once its elided instance arguments and the general-F reduction are supplied, is expected to type-check against Mathlib commit `0c1d5a7e` and to introduce no new axiom dependencies beyond those already present in the scalar Lagrange statement.

Reproducibility. Every figure in this paper is regenerable from public artefacts:

- The corpus-scale numbers $N = 707{,}053$, $r = 50.84\,\%$, and the profile count 23 are quoted verbatim from [1]; the source document is at Zenodo and the numbers are given in its abstract and tables.
- The direct-use count $D = 206$ was obtained by counting, in the Mathlib commit `0c1d5a7e`, the declarations whose #print axioms output contains Classical.choice; the same count is reproducible from the Mathlib source tree at that commit.
- The depth-5 tree in Section 6.1 is regenerable by invoking gen_tree.py on the edge dataset mathlib_edges.csv [8]:

```
python gen_tree.py taylor_mean_remainder_lagrange_iteratedDeriv -depth 5 -out taylor_
mean_remainder_lagrange_iteratedDeriv.tree.depth5.txt
```

The full tree has on the order of 10^5 nodes; the printed excerpt is chosen as described in Section 6.1.

- The scalar-theorem tree used implicitly throughout is regenerable by the same invocation on `taylor_mean_remainder_lagrange` without a depth bound and with `–only-thms` to prune definition/instance subtrees.
- The `#print axioms` output reported in Section 2 and Section 6.2 is regenerable by opening a Lean 4 session against Mathlib commit `0c1d5a7e` and running `#print axioms` on the two declarations named.

All arithmetic in the paper is elementary; no step requires numerical analysis beyond the four-quantity provenance table of Section 1. We invite the reader to substitute their own count for D and to re-run the two tree invocations above; no load-bearing claim in the paper depends on the numerical value of D or on the precise node budget of the printed excerpt.

References

[1] Larsen James Close. The Axiom Profile of Mathlib. Zenodo, 2026. DOI: 10.5281/zenodo.20710920. https://zenodo.org/records/20710921.

[2] DeepSeek. DeepSeek Chat. Conversational assistant used to assemble the exposition, cross-check the statement signatures against the Mathlib source, and prepare the LaTeX source of this paper. https://chat.deepseek.com.

[3] Lean community. Mathlib statistics. https://leanprover-community.github.io/mathlib_stats.html.

[4] The mathlib Community. The Lean mathematical library. Source repository, GitHub. https://github.com/leanprover-community/mathlib4.

[5] The mathlib Community. Mathlib4 documentation browser. Online reference for every declaration in Mathlib; used to locate the signatures and dependency edges of the theorems in this paper, and recommended as the entry point for analogous dependency-guided expositions of other theorems in the same library. https://leanprover-community.github.io/mathlib4_docs/.

[6] The mathlib Community. `Mathlib/Analysis/Calculus/Taylor.lean`. Mathlib4 source file containing the iterated Taylor–Lagrange theorem and the scalar reduction used in Section 5 and Section 6. Pinned to commit `0c1d5a7e`. https://github.com/leanprover-community/mathlib4/blob/0c1d5a7e/Mathlib/Analysis/Calculus/Taylor.lean.

[7] Xinze Li and Patrick Shafto. The Multinetwork of Mathlib. NeurIPS 2026 Datasets and Benchmarks Track, 2026. Under review. Dataset file: `mathlib_edges.csv`. https://huggingface.co/datasets/MathNetwork/MathlibGraph/resolve/main/mathlib_edges.csv.

[8] Xinze Li and Patrick Shafto. MathlibGraph: The Multinetwork of Mathlib. Hugging Face dataset, 2026. https://huggingface.co/datasets/MathNetwork/MathlibGraph.

[9] Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The Network Structure of Mathlib. arXiv:2604.24797, 2026. https://arxiv.org/abs/2604.24797.

[10] Freek Wiedijk. Formalizing 100 Theorems. In: The Seventeen Provers of the World, Springer LNAI 3600, 2006, pp. 101–113. List maintained online at https://www.cs.ru.nl/~freek/100/. Taylor's theorem appears as item 35 of the list.